

ARIA 블록암호 기반 HCTR2 운용모드의 성능 분석 및 최적화

엄시우*, 송민호*, 윤세영*, 서화정**

*한성대학교 (대학원생)

**한성대학교 (교수)

Performance Analysis and SIMD Optimization of ARIA-128 based HCTR2 on ARM

Si-Woo Eum*, Min-Ho Song*, Se-Young Yoon*, Hwa-Jeong Seo**

*Hansung University(Graduate student)

**Hansung University(Professor)

요 약

NIST는 기존 블록암호 운용모드의 한계를 극복하고 AEAD를 지원하기 위해 Accordion이라는 새로운 운용모드 표준화를 진행하고 있다. Accordion은 HCTR2 설계를 참조한 설계를 권장하고 있다. 본 논문에서는 국내 표준 블록암호인 ARIA 블록암호를 HCTR2에 적용하여 성능을 분석한다. 특히, HCTR2 구조의 핵심 연산 중 병렬 처리가 가능한 XCTR 부분을 병렬로 처리함으로써 성능 최적화했다. Apple M1 및 M4 Pro 환경에서 최적화된 ARIA-HCTR2의 성능을 OpenSSL의 GCM,CTR, CBC, ECB와 비교 분석하였다. 결과적으로 M4 기준 SIMD 최적화된 ARIA-HCTR2는 일반 구현 대비 약 6배 성능 향상을 보였으며, OpenSSL의 하드웨어 가속 최적화가 적용된 ARIA-GCM 보다도 약 28% 더 높은 성능을 보여준다. 이러한 결과를 통해 ARIA 기반 HCTR2가 차세대 AEAD 운용모드로서 강력한 잠재력을 가지고 있음을 확인하였다.

I. 서론

블록암호는 고정된 크기의 데이터 블록만 암호화할 수 있기 때문에, 임의 길이의 데이터를 안전하게 암호화하기 위해 블록암호 운용모드를 사용한다. 이러한 블록암호 운용모드를 통해 기밀성, 무결성, 인증 등의 추가적인 보안 요소를 제공할 수 있다. AEAD(Authenticated Encryption with Associated Data)는 암호화와 동시에 데이터의 진위성을 검증하는 인증 태그를 생성하여, 기밀성과 무결성을 한 번에 보장하는 암호화 방식이다. 대표적인 AEAD인 GCM(Galois/Counter Mode)은 높은 성능과 병렬 처리 이점으로 표준으로 자리 잡았으나, Nonce 재사용 시 치명적인 보안 취약점이 존재한다. 이러한 기존 운용모드들의 한계를 해결하고 개선하기 위해 NIST는 새로운 운용모드 표준화 프로젝트 'Accordion'을 발표했다[1, 2].

Accordion은 Google에서 제안한 HCTR2 설계를 주요 참조 모델로 한다[3]. HCTR2는 Tweakeable SPRP로, Nonce 재사용에도 안전하며 길이 보존 암호화를 지원한다. Accordion은 Acc128, Acc256, BBBAcc 세 개의 모드를 목표로 하며, 각 모드는 HCTR2의 변형을 통한 설계를 권고하고 있다.

본 연구는 HCTR2 설계를 국내 표준 128비트 블록암호인 ARIA에 적용하고, 구조적 특징을 활용한 성능 최적화를 수행한다. HCTR2의 핵심 연산은 카운터 기반 암호화를 수행하는 XCTR과 Polyval 해시로 구성되며, XCTR은 CTR 모드와 유사하게 본질적으로 병렬화가 가능한 구조이다. 이에 선행 연구된 ARIA의 ARM 기반 SIMD 병렬 구현을 HCTR2 최적화에 적용하고, 기존 블록암호 운용모드와의 성능 비교를 통해 HCTR2의 실용성을 검증하고자 한다.

II. 관련 연구

2.1 ARIA 블록암호

ARIA는 2004년 국가보안기술연구소 주도로 개발된 국내 표준 128비트 블록 암호이다. 2009년 국제표준화기구(ISO)와 국제 전기기술위원회(IEC)에 의해 국제 표준(ISO/IEC 18033-3)으로 채택되었다. 또한 2010년 국가표준(KS X 1213)으로 지정되어 국내외에서 공인된 암호 알고리즘으로 자리매김하였다[4].

ARIA는 Substitution-Permutation Network(SPN) 구조를 기반으로 설계되었으며, 128/192/256비트의 키 길이를 지원한다. AES와 유사한 연산 구조를 가지고 있으나, 두 개의 서로 다른 8x8 S-Box를 교대로 사용하는 Involutional SPN 구조로 동작되는 것이 특징이다.

2.2 블록암호 운용모드 및 OpenSSL

ECB, CBC, CTR 등은 기밀성을 제공하는 전통적인 운용모드이다. GCM은 CTR 모드와 GHASH를 결합하여 AEAD를 지원하며, 최근 TLS 1.3에 오면서 CBC 모드와 같은 기존 기밀성만을 제공하는 운용모드는 제거하고 GCM 등 AEAD 모드만을 표준으로 채택하여 사용되고 있다.

OpenSSL은 SSL/TLS 프로토콜과 다양한 암호 알고리즘을 구현한 오픈소스 암호화 라이브러리로, 산업 표준으로 널리 사용되고 있다[5]. OpenSSL은 speed 명령어를 통해 암호 알고리즘의 성능을 측정하는 벤치마크 기능을 제공하며, 이는 CPU의 하드웨어 가속 기능을 활용한 최적화된 성능을 측정한다. 본 연구에서는 'OPENSSL_armcap=""~0x0' 옵션을 통해 하드웨어 가속을 비활성화함으로써 순수 C 구현 성능과 최적화된 성능 두 가지를 비교군으로 활용한다.

2.3 HCTR2 구조

HCTR2는 [그림 1]과 같이 Tweak과 평문을

입력받아 길이 보존 암호화를 수행한다. 핵심 구조는 크게 두 부분으로 구성된다.

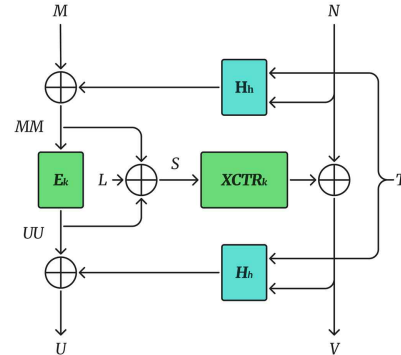


그림 1. HCTR2 구조

첫째, XCTR(XORed-Counter) 계층이다. XCTR은 CTR 모드와 유사한 구조로 동작하지만, CTR 모드가 IV와 Counter 값을 연접(concatenation)하여 블록암호의 입력으로 사용하는 반면, XCTR은 IV에 Counter 값을 XOR한 결과를 입력으로 사용한다. 이러한 차이를 제외하면 평문의 대부분을 CTR 모드와 동일하게 병렬 처리할 수 있다.

둘째, Polyval 해시 연산이다. Polyval은 GCM의 GHASH와 유사한 $GF(2^{128})$ 상의 유니버설 해시 함수로, Tweak 처리 및 무결성 검증에 활용된다.

본 연구의 핵심은 ARIA 블록암호 연산을 병렬 수행이 가능한 XCTR 부분에 SIMD(NEON) 최적화를 적용하여 성능을 최적화하는 것이다.

III. ARIA-HCTR2 최적화 구현

3.1 Reference 참조 ARIA-HCTR2 구현

본 논문에서는 HCTR2의 Reference Implementation을 기반으로 블록암호를 AES에서 ARIA로 교체하여 구현하였다[6]. HCTR2는 XCTR과 POLYVAL을 결합한 Wide-Block 암호 구조로 다음과 같은 형태로 동작한다.

Wide-Block이란 일반 블록암호의 고정 크기(128비트)를 넘어 수 KB의 데이터를 하나의 단위로 처리하는 구조를 의미한다. 이는 메시지

의 한 비트 변경이 전체 암호문에 영향을 준다.

HCTR2는 블록암호 운용모드이기 때문에 내부 블록암호 알고리즘과 독립적으로 설계되어 있다. 따라서 Reference 구현에서 블록 암호만 변경함으로써 ARIA 블록암호를 적용할 수 있다. 구체적으로 XCTR의 카운터 모드 암호화 부분과 POLYVAL의 subkey 생성 부분에서 AES 호출을 ARIA 함수로 교체하였다. 이를 통해 ARIA-HCTR2의 기본 구현을 완성하였으며, 추가적인 최적화 작업을 수행하였다.

3.2 ARIA-HCTR2 최적화 구현

성능을 다각도로 분석하기 위해 Reference 구현에서 ARIA 블록암호로 교체한 일반 구현과 SIMD를 활용한 최적화 구현을 활용하였다.

3.2.1 일반 구현

첫 번째 구현에서는 최적화가 적용되지 않은 표준 ARIA 구현을 적용한다. 이는 기본적인 성능 기준을 측정하고, 최적화에 따른 성능 향상을 정량적으로 비교하기 위함이다.

3.2.2 SIMD 최적화 구현

두 번째 구현에서는 ARM NEON SIMD 명령어로 최적 구현된 ARIA 구현을 적용한다. HCTR2의 핵심 연산인 XCTR은 다음과 같은 구조로 연산된다.

$$XCTR_k(S) = E_k(S \oplus bin(1)) \parallel E_k(S \oplus bin(2)) \parallel E_k(S \oplus bin(3)) \parallel \dots$$

여기서 각 블록의 암호화 $E_k()$ 는 서로 독립적으로 연산 가능하다. 따라서 여러 블록을 동시에 처리하는 SIMD 명령어에 매우 적합하다. 본 논문에서는 [7]에서 제안된 ARIA의 NEON 기반 병렬 구현을 적용하여 HCTR2의 XCTR 연산에 통합함으로써 한 번의 루프에서 여러 ARIA 블록이 병렬로 처리되도록 최적화하였다.

[7]에서의 핵심 전략은 다음과 같다. 첫째, ARIA의 확산 연산과 치환 연산에 최적화된 두 개의 데이터 상태를 정의하였다. 둘째, TRN1과 TRN2와 같은 벡터 명령어를 활용함으로써 메

모리 접근 없이 치환 연산을 최적화하였다. 마지막으로 연산 순서를 조정함으로써 두 개의 Sbox를 불러오는 과정을 최소화하여 S-box 로드 횟수를 최소화하였다. 자세한 구현 과정은 이전 연구에서 참조 가능하다.

IV. 성능 평가 및 분석

	Env.1	Env.2
CPU	Apple M4 Pro	Apple M1
OpenSSL	3.1.4	3.5.2
Compiler	clang 17.0.0	clang 17.0.0

표 1. 성능 측정 환경

본 논문에서는 ARM NEON SIMD 최적화된 구현을 활용하였기 때문에 ARM 아키텍처를 사용하는 Apple M시리즈 환경에서 성능 측정하여 분석을 수행하였다.

운용모드	M1 속도(MB/s)	M4 Pro 속도(MB/s)
ECB	205.85	306.77
CBC	178.94	256.74
CTR	180.91	268.20
GCM	119.44	184.67
HCTR2	50.44	65.58
ECB*	204.48	306.63
CBC*	177.76	258.81
CTR*	180.75	270.81
GCM*	198.16	309.98
HCTR2*	233.95	396.75

표 2. 성능 측정 결과(단위: MB/s, *: 최적화 구현)

HCTR2의 SIMD 최적화는 두 환경 모두에서 큰 성능 향상을 보여주었다. M4 Pro에서는 일반 구현(65.58) 대비 SIMD 구현(396.75)로 약 6배의 성능 향상을 보였다. M1에도 일반 구현(50.44) 대비 SIMD 구현(233.95)로 약 4배의 성능 향상을 보였다.

이를 통해 HCTR2의 XCTR 구조에서 SIMD 기반 병렬 구현을 통한 최적화 통해 높은 성능 향상을 보여줄 수 있음을 확인할 수 있다. NEON 명령어를 통한 벡터화된 블록 암호 연산

이 성능 향상의 핵심 요인으로 작용했다.

OpenSSL 측정 결과, GCM 모드에서만 하드웨어 가속 여부에 따른 큰 성능 차이가 관찰되었다. 반면, CTR, CBC, ECB 모드는 가속 여부에 따른 성능 차이가 1% 미만으로 거의 없었다. 이는 Apple M1/M4 칩셋이 GCM의 핵심 연산인 GF(2^{128}) 상의 곱셈(GHASH)을 위한 PMULL 명령어를 탑재하고 있으나, ARIA 블록 암호 자체에 대한 전용 하드웨어 가속은 지원하지 않음을 시사한다.

본 연구의 핵심은 최신 AEAD 후보인 HCTR2와 표준 AEAD인 GCM의 성능 비교이다. 각 환경에서 가장 최적화된 구현인 HCTR2-SIMD와 GCM 하드웨어 가속을 비교한 결과, HCTR2-SIMD가 M4 Pro에서 약 28.0% 더 빠른 성능을 보였다. 마찬가지로 M1에서도 약 18.1% 더 빠른 성능을 보였다.

이러한 결과를 통해 AEAD 모드인 HCTR2가 GCM보다 높은 성능을 보임으로써, GCM 모드에서의 문제점을 보완한 Accordion 모드가 충분한 성능을 보여줄 수 있음을 확인하였다.

V. 결론

본 논문에서는 NIST에서 진행하고 있는 새로운 AEAD 표준화 Accordion 운용모드의 기반 기술인 HCTR2에 국내 표준 블록암호 ARIA-128을 적용하고, 최신 ARM 프로세서인 Apple M1, M4 Pro 환경에서 성능을 최적화 및 분석했다. HCTR2의 XCTR 연산 부분을 병렬 처리를 함으로써 일반 구현 대비 M1에서 약 4배, M4 Pro에서 약 6배의 성능 향상을 확인했다. 또한, SIMD 최적화된 ARIA-HCTR2는 하드웨어 가속을 포함한 OpenSSL의 ARIA-GCM 보다 M1에서 약 18%, M4 Pro에서 약 28% 빠른 성능을 보여준다. 이는 HCTR2이 GCM의 성능과 대등할 수 있으며 차세대 AEAD 운용모드로 대체될 수 있음을 시사한다. 향후 연구로는 HCTR2의 GPU를 활용한 최적화와 Polyval 해시 부분에 대한 최적화를 통해 ARIA-HCTR2의 효율성에 대한 더 자세한 분석을 제안한다.

VI. Acknowledgment

This work was supported by Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIT) (No.2018-0-00264, Research on Blockchain Security Technology for IoT Services, 20%) and this work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2025-02306395, Development and Demonstration of PQC-Based Joint Certificate PKI Technology, 40%) and this work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-25394739, Development of Security Enhancement Technology for Industrial Control Systems Based on S/HBOM Supply Chain Protection, 40%).

[참고문헌]

- [1] D.S.Milojicic, V.Kalogeraki, R.Lukose,K.Nagaraja, J.Pruyne, B.Richard, S.Rollins and Z.Xu, Peer to Peer Computing, HP Laboratories Palo Alto HPL-2002-57, March, 2002.
- [1] Mouha N, Dworkin M (2023) Report on the Block Cipher Modes of Operation in the NIST SP 800-38 Series (initial public draft), NISTIR 8459. <https://doi.org/10.6028/NIST.IR.8459.ipd>.
- [2] Chen, Yu Long, et al. "Proposal of requirements for an accordion mode." Discussion Draft for the NIST Accordion Mode Workshop 2024. 2024.
- [3] Crowley, Paul, Nathan Huckleberry,

and Eric Biggers. "Length-preserving encryption with HCTR2." Cryptology ePrint Archive (2021).

- [4] Kwon, Daesung, et al. "New block cipher: ARIA." International conference on information security and cryptology. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003.
- [5] OpenSSL Software Foundation, "OpenSSL," 2025. [Online]. Available: <https://www.openssl.org/>. [Accessed: Oct. 26, 2025].
- [6] Google, "HCTR2 Implementation," GitHub. [Online]. Available: <https://github.com/google/hctr2>. [Accessed: Oct. 26, 2025].
- [7] Eum, Siwoo, et al. "Parallel implementations of ARIA on ARM processors and graphics processing unit." Applied Sciences 12.23 (2022): 12246.